Universidad San Francisco de Quito

Colegio de Ciencias e Ingenierías

Redes de Computadoras

CTF Layer 2 Security

Guía de Ejecución Paso a Paso

Equipo:

Julian Leon
James Soto
Mauricio Mantilla

Marzo 2026

Repositorio: https://github.com/jjjulianleon/ProyectoFinalRedes

Índice

1. Introducción **2**

2. Prerrequisitos **2**
2.1. Máquina Virtual con Kali Linux 2
2.2. Docker + Docker Compose 2
2.3. Git 3

3. Pasos de Ejecución **3**
3.1. Paso 1: Clonar el repositorio 3
3.2. Paso 2: Crear el archivo `.env` 3
3.3. Paso 3: Generar certificados SSL de Wazuh 3
3.4. Paso 4: Configurar parámetros del sistema e iniciar contenedores 3
3.5. Paso 5: Verificar el entorno 4
3.6. Paso 6: Configurar CTFd 4
3.7. Paso 7: Configurar integración Wazuh + Suricata 4
3.8. Paso 8: Ejecutar la demo del CTF 4
3.9. Paso 9: Ver resultados 5
3.9.1. Directorio de evidencia (`evidence_*/`) 5
3.9.2. Interfaces web 5
3.10. Paso 10: Verificar alertas en Wazuh Dashboard 5
3.11. Paso 11: Analizar capturas en Wireshark 7

4. Ejecución Manual (Modo Interactivo) **8**
4.1. Red Team (ataque) 8
4.2. Blue Team (defensa) 9

5. Arquitectura de Red **9**

6. Solución de Problemas **9**

7. Comandos Útiles **10**

1. Introducción

Este documento describe cómo clonar, configurar y ejecutar el proyecto **CTF Layer 2 Security** en una máquina local. El entorno completo corre sobre Docker y simula una red con contenedores para practicar ataques y defensas de Capa 2 (ARP Spoofing, MAC Flooding) con monitoreo mediante Suricata y Wazuh.

2. Prerrequisitos

2.1. Máquina Virtual con Kali Linux

Se recomienda una VM de Kali Linux Rolling (la versión más reciente). Opciones de hipervisor:

- **Linux:** virt-manager (recomendado)
- **Windows/Mac:** VirtualBox o VMware

Requisitos mínimos de la VM:

- **RAM:** 8 GB asignados a la VM
- **Disco:** al menos 30 GB
- **Sistema:** Kali Rolling (latest)

2.2. Docker + Docker Compose

Versiones probadas: Docker CE v27.5.1 y Docker Compose plugin v5.1.0.

Instalación en Kali:

```
sudo apt update
sudo apt install docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

El plugin de Docker Compose no está en los repositorios de Kali, por lo que se debe instalar manualmente:

```
DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL https://github.com/docker/compose/releases/latest/
   download/docker-compose-linux-x86_64 \
  -o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose
```

Verificar la instalación:

```
docker compose version
```

Importante: después de ejecutar `usermod`, cerrar sesión y volver a iniciarla para que el grupo `docker` surta efecto. Si no, usar `sudo` antes de cada comando `docker`.

2.3. Git

```
sudo apt install git
```

3. Pasos de Ejecución

3.1. Paso 1: Clonar el repositorio

```
git clone https://github.com/jjjulianleon/ProyectoFinalRedes.git
cd ProyectoFinalRedes/ctf-layer2-security
```

3.2. Paso 2: Crear el archivo `.env`

Crear un archivo llamado `.env` dentro del directorio `ctf-layer2-security/` con el siguiente contenido:

```
FLAG1=FLAG{arp_spoof_mitm_captured}
FLAG2=FLAG{mac_flood_broadcast_leak}
FLAG3=FLAG{plaintext_credentials_exposed}
```

Se puede crear con un editor de texto o directamente desde la terminal:

```
cat > .env << 'EOF'
FLAG1=FLAG{arp_spoof_mitm_captured}
FLAG2=FLAG{mac_flood_broadcast_leak}
FLAG3=FLAG{plaintext_credentials_exposed}
EOF
```

3.3. Paso 3: Generar certificados SSL de Wazuh

```
cd containers/wazuh
sudo docker compose -f generate-certs-compose.yml run --rm
   generator
cd ../..
```

3.4. Paso 4: Configurar parámetros del sistema e iniciar contenedores

```
sudo sysctl -w vm.max_map_count=262144
sudo docker compose up -d --build
```

Esto construye e inicia los **13 contenedores**. La primera vez tarda aproximadamente **10 minutos** en descargar las imágenes.

3.5. Paso 5: Verificar el entorno

```
sudo bash scripts/verify_environment.sh
```

Las **31 verificaciones** deben pasar correctamente.

3.6. Paso 6: Configurar CTFd

1. Abrir `http://localhost:8000` en el navegador.
2. Completar el asistente de configuración:
   - **Team name:** admin
   - **Email:** admin@ctf.local
   - **Password:** (elegir uno)
   - **Modo:** Team Mode
3. Generar un token de API: `Settings` → `Access Tokens` → `Generate`.
4. Crear los retos (challenges):

```
# Instalar requests en victim1 y copiar script de setup
sudo docker exec victim1 pip3 install requests 2>/dev/null
sudo docker cp containers/ctfd/config/setup_ctfd.py victim1:/tmp/
sudo docker exec victim1 python3 /tmp/setup_ctfd.py <TU_API_TOKEN
    >
```

Reemplazar `<TU_API_TOKEN>` con el token generado en el paso anterior.

3.7. Paso 7: Configurar integración Wazuh + Suricata

```
sudo bash scripts/setup_wazuh_suricata.sh
```

3.8. Paso 8: Ejecutar la demo del CTF

```
sudo bash scripts/run_ctf_demo.sh
```

Este script tarda aproximadamente **2 minutos** y realiza lo siguiente:

1. Inicia el monitoreo del Blue Team.
2. Ejecuta ARP Spoofing sobre las 3 víctimas.
3. Captura las 3 flags.
4. Ejecuta MAC Flooding.
5. Restaura las tablas ARP.
6. Recopila toda la evidencia en el directorio `evidence_YYYYMMDD_HHMMSS/`.

3.9. Paso 9: Ver resultados

3.9.1. Directorio de evidencia (`evidence_*/`)

- `arp_alerts.log` – Detecciones de ARP spoofing
- `mac_flood_alerts.log` – Detecciones de MAC flooding
- `suricata_fast.log` – Alertas del NIDS Suricata
- `captura_ctf.pcap` – Captura completa de tráfico (abrir con Wireshark)
- `victim*_response.html` / `victim*_credentials.txt` – Datos capturados

3.9.2. Interfaces web

- **CTFd:** http://localhost:8000
- **Wazuh Dashboard:** https://localhost:5601
  - Usuario: `admin`
  - Contraseña: `SecretPassword`

3.10. Paso 10: Verificar alertas en Wazuh Dashboard

1. Abrir https://localhost:5601 en el navegador.
2. Ingresar credenciales: `admin` / `SecretPassword`.
3. En el panel principal (Overview) se verán las alertas clasificadas por severidad.
4. Ir a **Threat Hunting** en el menú lateral.
5. En la barra de búsqueda escribir `suricata` para filtrar las alertas del NIDS.

6. Se deberían ver las alertas de flags en texto plano y credenciales detectadas.

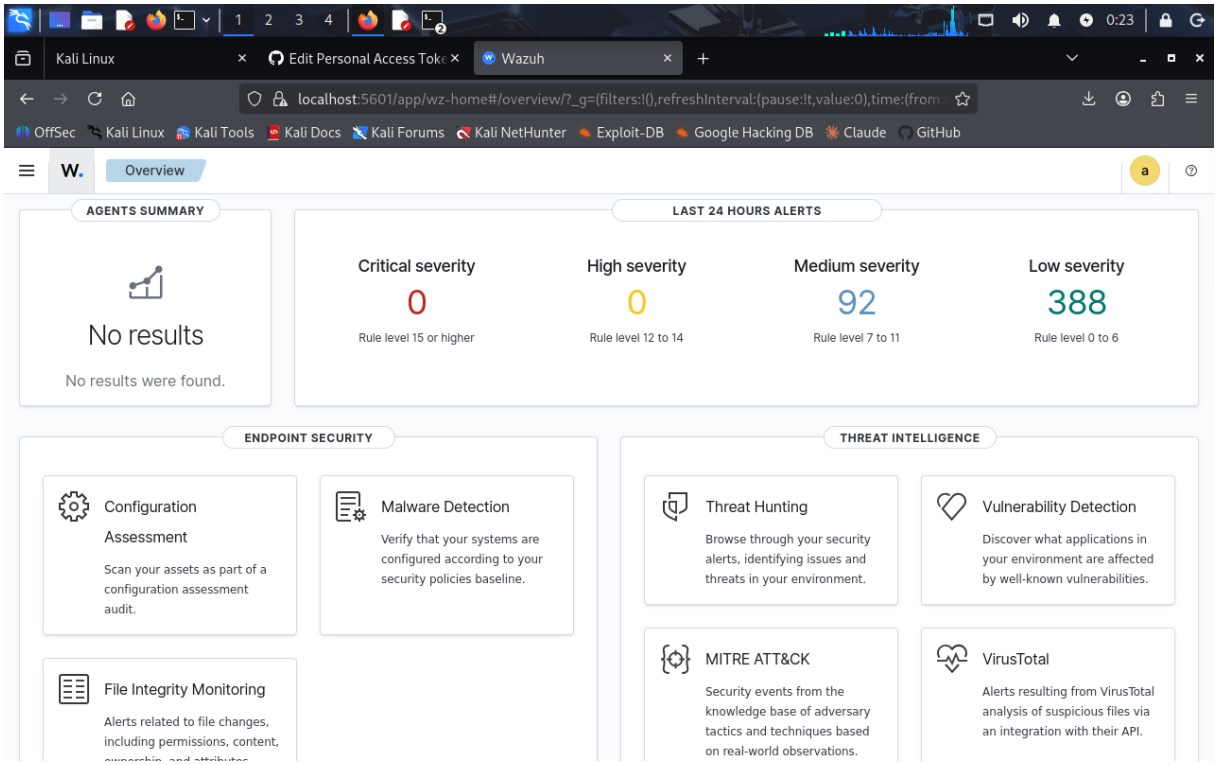

Figura 1: Panel Overview de Wazuh con alertas generadas durante el CTF.

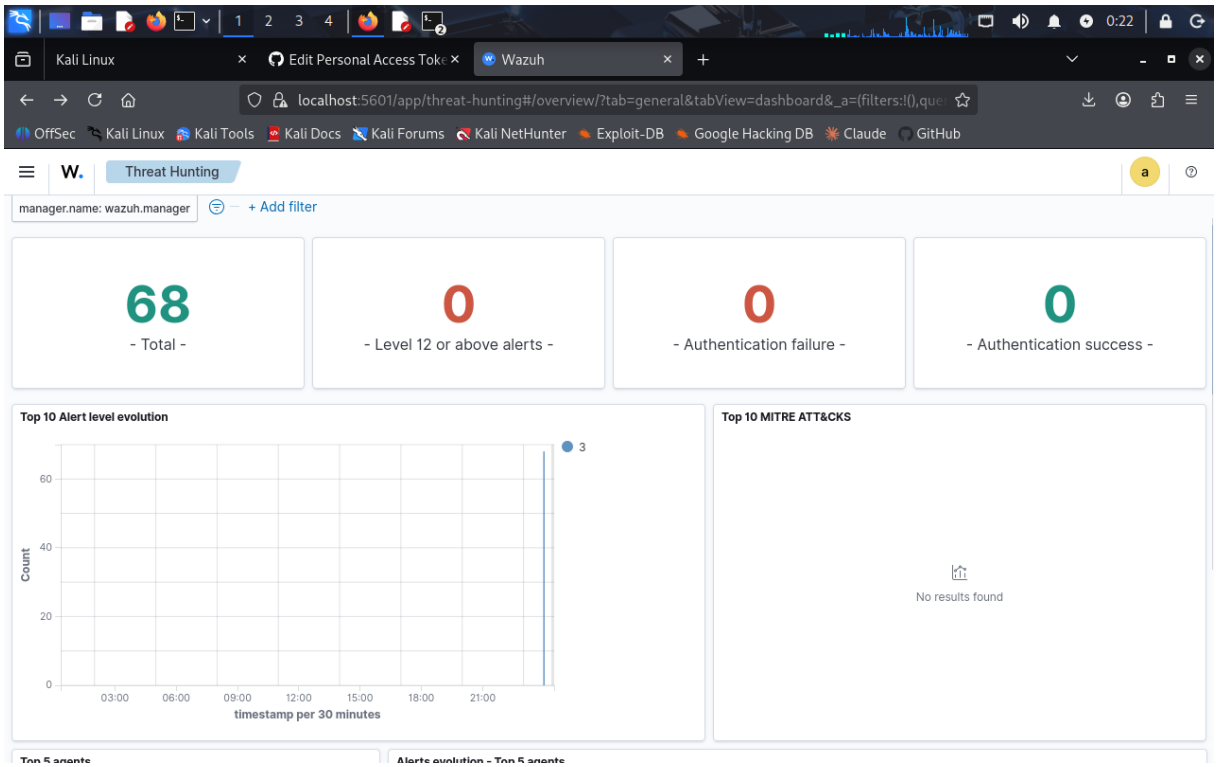

Figura 2: Módulo Threat Hunting filtrando alertas de Suricata.

3.11. Paso 11: Analizar capturas en Wireshark

Abrir la captura de tráfico generada:

```
sudo wireshark evidence_*/captura_ctf.pcap
```

Filtros útiles en Wireshark:

- **Ver ARP Spoofing:** `arp.opcode == 2 && arp.src.hw_mac == 02:42:ac:14:00:64`
- **Ver MAC Flooding:** `arp.opcode == 1` (buscar MACs aleatorias al final de la captura)
- **Ver todo el ARP:** `arp`

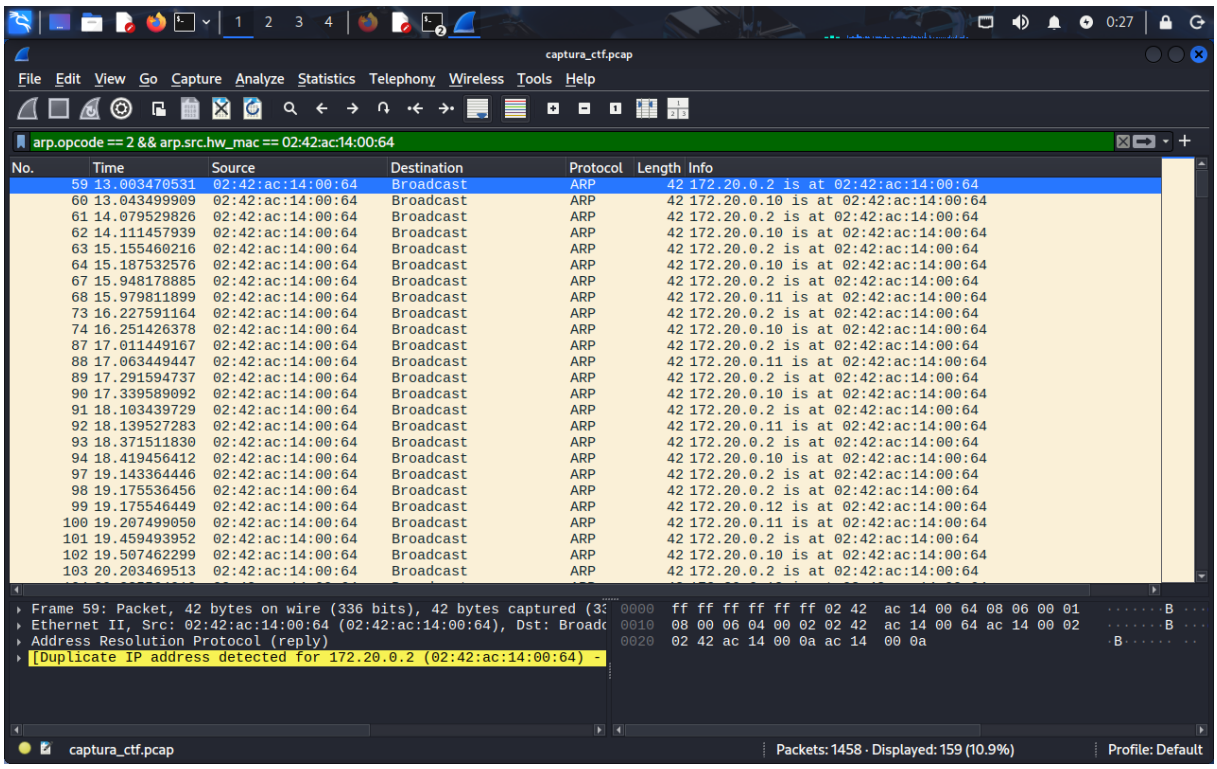

Figura 3: ARP Replies falsos del atacante suplantando el gateway.

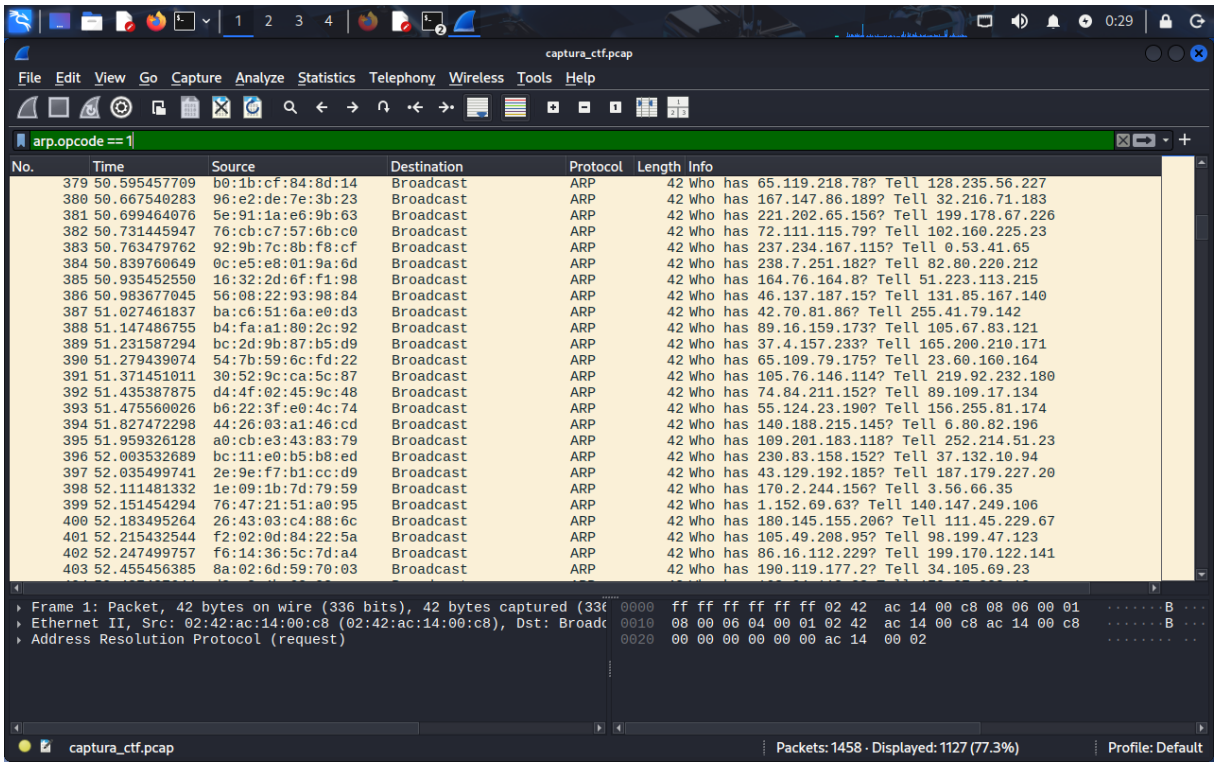

Figura 4: ARP Requests masivos con MACs aleatorias (MAC Flooding).

4. Ejecución Manual (Modo Interactivo)

Para una ejecución más detallada y controlada, se pueden ejecutar los comandos manualmente en terminales separadas.

4.1. Red Team (ataque)

```
# Terminal 1: ARP Spoofing
sudo docker exec -it redteam \
  python3 /tools/arp_spoof.py -t 172.20.0.12 -g 172.20.0.2 --
     broadcast

# Terminal 2: Capturar flags
sudo docker exec -it redteam python3 /tools/capture_flags.py

# Terminal 3: MAC Flooding
sudo docker exec -it redteam python3 /tools/mac_flood.py -c 5000

# Enviar flags al CTFd
sudo docker exec -it redteam \
  python3 /tools/submit_flag.py --list --token <TOKEN>
sudo docker exec -it redteam \
```

```
  python3 /tools/submit_flag.py -f "FLAG{...}" -c 1 --token <
     TOKEN>
```

4.2. Blue Team (defensa)

```
# Terminal 1: Monitor ARP
sudo docker exec -it blueteam \
  python3 /tools/arp_monitor.py --known 172.20.0.2 172.20.0.10
     172.20.0.11 172.20.0.12

# Terminal 2: Detector de anomalias MAC
sudo docker exec -it blueteam python3 /tools/mac_anomaly_detector
   .py

# Terminal 3: Restaurar ARP (despues del ataque)
sudo docker exec -it blueteam python3 /tools/arp_restore.py --
   continuous
```

5. Arquitectura de Red

La siguiente tabla muestra los contenedores, sus direcciones IP y roles dentro del entorno:

Contenedor	**IP**	**Rol**
Gateway	172.20.0.2	Router / objetivo de ataques
Victim 1	172.20.0.10	Servidor HTTP (flag en HTML)
Victim 2	172.20.0.11	Servidor de archivos (flag en archivo)
Victim 3	172.20.0.12	Agente de monitoreo (flag en tráfico)
Red Team	172.20.0.100	Atacante
Blue Team	172.20.0.200	Defensor
Suricata	(red gateway)	NIDS (Sistema de Detección de Intrusos)
Wazuh Manager	172.20.0.240	SIEM – Gestor
Wazuh Indexer	172.20.0.241	SIEM – Almacenamiento de logs
Wazuh Dashboard	172.20.0.242	SIEM – Interfaz web
CTFd	172.20.0.250	Plataforma CTF
CTFd DB	172.20.0.251	Base de datos (MariaDB)
CTFd Cache	172.20.0.252	Caché (Redis)

Cuadro 1: Contenedores y direcciones IP del entorno CTF

6. Solución de Problemas

`permission denied` al usar Docker
Ejecutar `sudo usermod -aG docker $USER` y luego cerrar sesión y volver a iniciarla. Alternativamente, usar `sudo` antes de cada comando `docker`.

Error de `vm.max_map_count` (Wazuh)
Ejecutar antes de iniciar los contenedores:

```
sudo sysctl -w vm.max_map_count=262144
```

Este parámetro se pierde al reiniciar la máquina; hay que ejecutarlo de nuevo después de cada reboot.

Los contenedores no inician
Verificar la RAM disponible con `free -h`. Se necesitan al menos **6 GB libres**.

CTFd no es accesible
Esperar 30 segundos después del arranque. Revisar logs con:

```
sudo docker logs ctfd
```

Wazuh Dashboard muestra error 503
Esperar 2–3 minutos para que el indexer se inicialice. Revisar logs con:

```
sudo docker logs wazuh-indexer
```

`docker-compose` no encontrado
En Kali, el comando es `docker compose` (con espacio, no guión). Instalar el plugin manualmente como se indica en la sección de prerrequisitos.

7. Comandos Útiles

```
# Ver estado de todos los contenedores
sudo docker ps --format "table {{.Names}}\t{{.Status}}"

# Reconstruir todo desde cero
sudo docker compose down -v && sudo docker compose up -d --build

# Ver logs de un contenedor
sudo docker logs <nombre_contenedor> 2>&1 | tail -20

# Entrar a un contenedor
sudo docker exec -it redteam bash
sudo docker exec -it blueteam bash

# Detener todo
```

```
sudo docker compose down
```